DocQuorum: controller, readers and tools

Automatic execution • bounded adaptive reader • one selected JSON

1 Inputs

- PDF / image
- Schema / template
- Optional task

Input evidence →

2 Prepare & route

PYTHON CONTROLLER

Normalize input; inspect text, pages and layout

Run MinerU for scans when available

Pillow • Poppler • layout helpers • MinerU

Prepared evidence →

3 Run reader(s)

MODEL CALLS

P / M / O / L

P M O L

See reader components below

Candidate JSONs →

4 Check each candidate

DETERMINISTIC CODE

Schema + source evidence

Repair that candidate only
No model call

Checked candidate JSONs →

5 Select one

DETERMINISTIC CODE

Route policy below

Quorum when needed: compare pairs; highest mean agreement

Keep one whole candidate

One JSON →

6 Return

JSON + selection report

Restore template shape if supplied

Reader components: who reads, what they use, what they produce

Same adaptive reader agent, two evidence modes

Primary reader (P)

Adaptive agent

P

Evidence: PDF text + permitted page images

Tools: Poppler text/rendering; schema + chunk helpers; extraction-model API

Output: candidate P

Parse-based reader (M)

Same adaptive agent

M

Evidence: cached MinerU text / HTML; images when the plan permits

Tools: MinerU cache; schema + chunk helpers; extraction-model API

Output: candidate M

Separate runs and candidate state; shared source evidence, no shared answers.

Scheduled workflow

Column reader (L)

L

Detect columns → crop → read top to bottom

Tools: Poppler rendering; column detector / cropper; extraction-model API

Output: layout candidate L

Scheduled workflow

Ruled-table reader (L)

L

Detect ruled rows → stack header + row band → read fixed row count

Tools: Poppler rendering; row-band detector; extraction-model API

Output: layout candidate L

Scheduled workflow

One-shot reader (O)

O

Whole document in one extraction call

Tools: ExtractBench provider; page rendering; extraction-model API

Output: candidate O

Tools are called by Python, not chosen autonomously by the LLM

Poppler [CODE] → page count • PDF text • page images

MinerU parser [MODEL] → cached text blocks + HTML tables

Schema / row / layout helpers [CODE] → scopes • chunks • crops • row counts

Extraction model via API [MODEL] → schema-guided JSON replies

Inside P / M: bounded agentic loop

1. **PLAN** [code] — Choose eligible scope
2. **ACT** [model] — Read selected evidence
3. **OBSERVE** [code] — Truncated? Missing keys? Empty despite row evidence?
4. **Retry needed?** — Yes → 5 ADAPT [code]: Increase budget; require keys; split or reduce scope → Observed reply changes next call → back to 1 PLAN
4. **Retry needed?** — No → Return candidate

Scopes: whole document / schema groups / text windows / scalar + list chunks

Python holds state and enforces retry rules; the LLM extracts values.

Which readers run, and how is one kept?

Document route	Readers attempted	Selection
Specialized layout	Column **OR** ruled (L)	**Keep L**
Text available	**Primary (P)**	**Keep P**
No text, ≤12 pages	P + M* + O	One → keep; multiple → quorum
No text, >12 pages	P then M*	Prefer retained M; otherwise P

Check chains (deterministic code)

- **P:** repairs → output checks
- **M:** spacing → repairs → output checks
- **L / O:** output checks

* M needs cached parse. Successful paths shown.

All stages after input submission are automatic. Agentic behavior is the bounded P/M feedback loop. No answer keys; no field-level candidate fusion.